Solution: Redundant Connection

Problem Statement

Given an undirected graph containing `1` to `n` nodes. The graph is represented by a 2D array of `edges`, where **edges[i] = [a_i, b_i]**, represents an edge between a_i, and b_i.

Identify one edge that, if removed, will turn the graph into a tree.

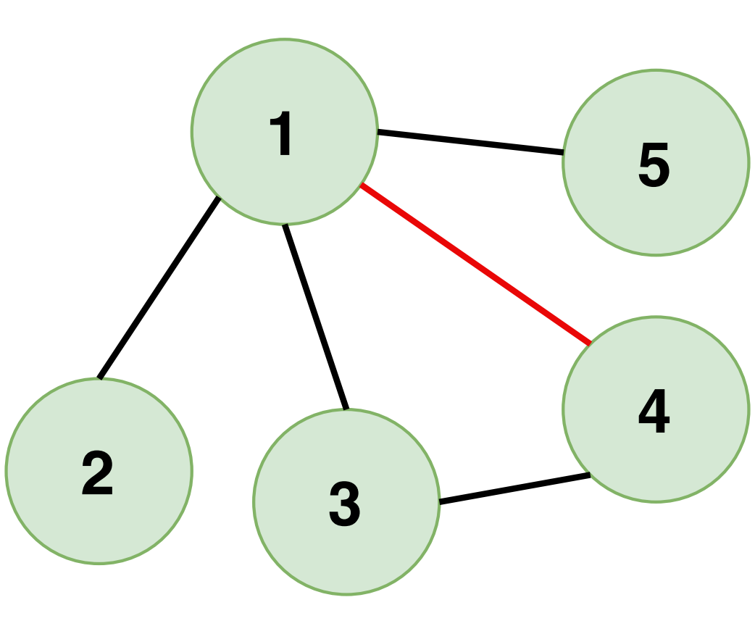
A tree is a graph that is connected and has no cycles.

Assume that the graph is always reducible to a tree by removing just one edge.

If there are multiple answers, return the edge that occurs last in the input.

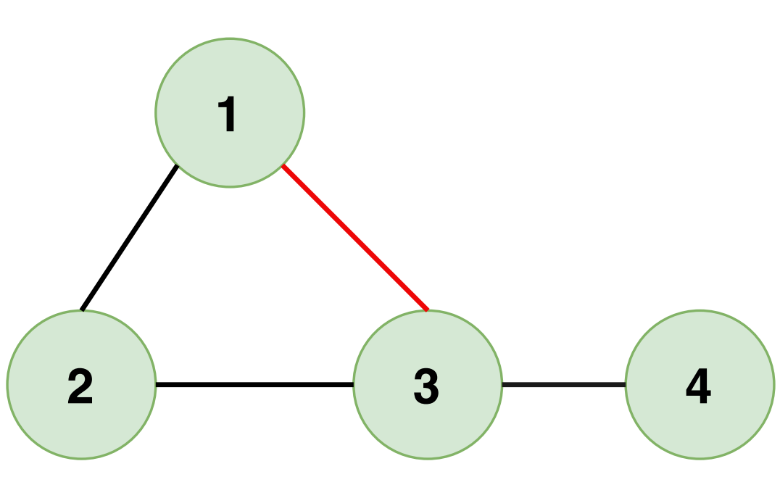
Examples

1. Example 1:
   - **Input**: `[[1,2], [1,3], [3,4], [1,4], [1,5]]`



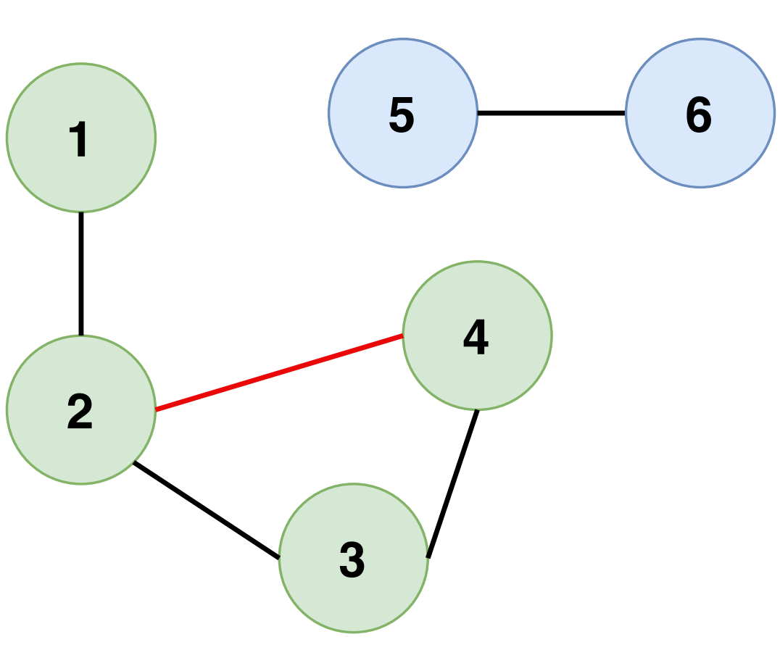
- **Expected Output**: `[1,4]`
- **Justification**: The edge `[1,4]` is redundant because removing it will eliminate the cycle `1–3–4–1` while keeping the graph connected.

2. Example 2:
   - **Input**: `[[1,2], [2,3], [3,1], [3,4]]`



- **Expected Output**: `[3,1]`
- **Justification**: The edge `[3,1]` creates a cycle `1–2–3–1`. Removing it leaves a tree with no cycles.

3. Example 3:
   - **Input**: `[[1,2], [2,3], [3,4], [4,2], [5,6]]`



- **Expected Output**: `[4,2]`
- **Justification**: The edge `[4,2]` adds a cycle `2–3–4–2` in one part of the graph. Removing this edge breaks the cycle, and the remaining graph is a tree.

Solution

The solution to this problem employs the Union-Find algorithm, which is effective for cycle detection in graphs. Initially, each node is considered as a separate set. As we iterate through the edges of the graph, we use the Union-Find approach to determine if the nodes connected by the current edge are already part of the same set. If they are, it indicates the presence of a cycle, and this edge is the redundant one. Otherwise, unite these two sets, meaning connect the nodes without forming a cycle. This approach focuses on progressively merging nodes into larger sets while keeping an eye out for the edge that connects nodes already in the same set.

Here's how we'll apply it:

1. Initialize an array to represent each node's parent. Initially, each node is its own parent, indicating that they are each in their own separate sets.
2. Iterate through the list of edges. For each edge, apply the "Find" operation to both nodes. If the nodes have different parents, they belong to different sets, and we can union them by pointing the parent of one to the other.
3. If the "Find" operation reveals that the two nodes already share a parent, then they're in the same set, meaning the edge under consideration creates a cycle. This edge is the redundant connection we're looking for.
4. The process ends once we find the redundant connection or after processing all edges.

This approach works as it precisely detects cycles in the way trees should not have them. It's efficient because the "Union" and "Find" operations can be optimized to almost constant time complexity with path compression and union by rank.

Algorithm Walkthrough:

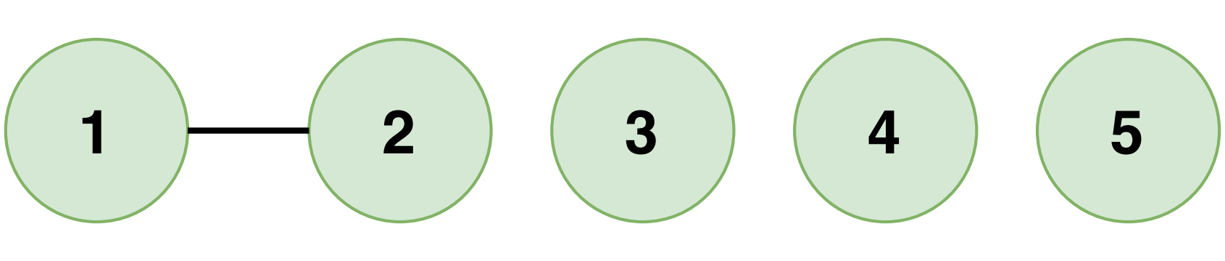
Consider the input [[1,2], [1,3], [3,4], [1,4], [1,5]].

- Initialize the parent array where `parent[i] = i`.

Element	1	2	3	4	5
Parent	1	2	3	4	5

- Consider the edge `[1,2]`:
  - Find the parents of `1` and `2`. Since they're different, union them by making one the parent of the other.

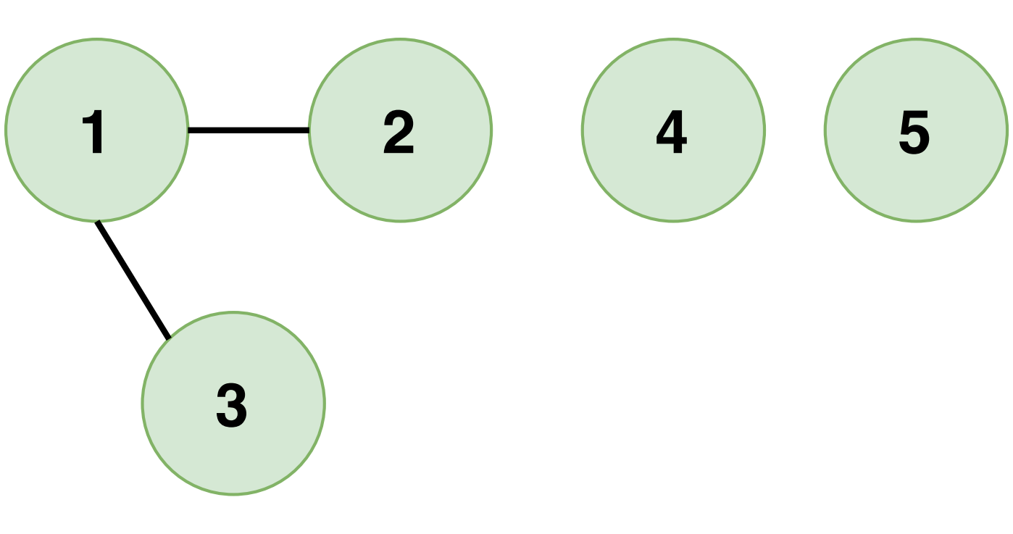
Union 1 and 2



Element	1	2	3	4	5
Parent	1	1	3	4	5

- Next, consider the edge `[1,3]`:
  - Their parents are different, so union them.

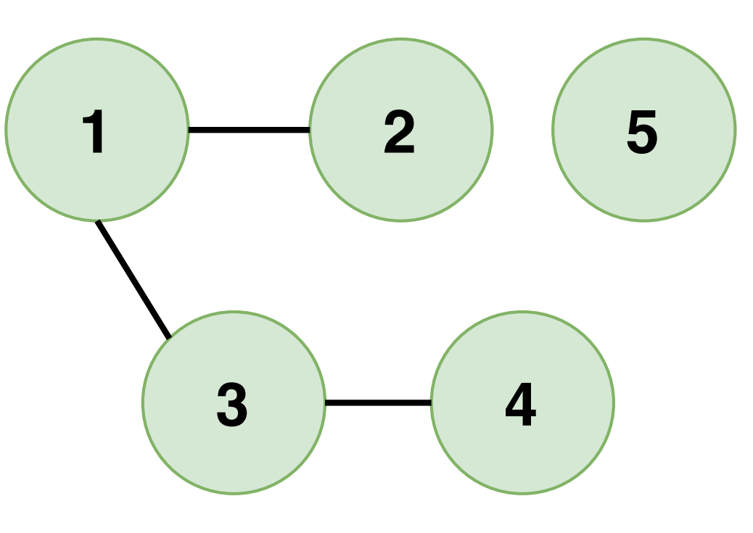
Union 1 and 3



Element	1	2	3	4	5
Parent	1	1	1	4	5

- Now, consider the edge `[3,4]`:
  - Again, different parents, so union them.

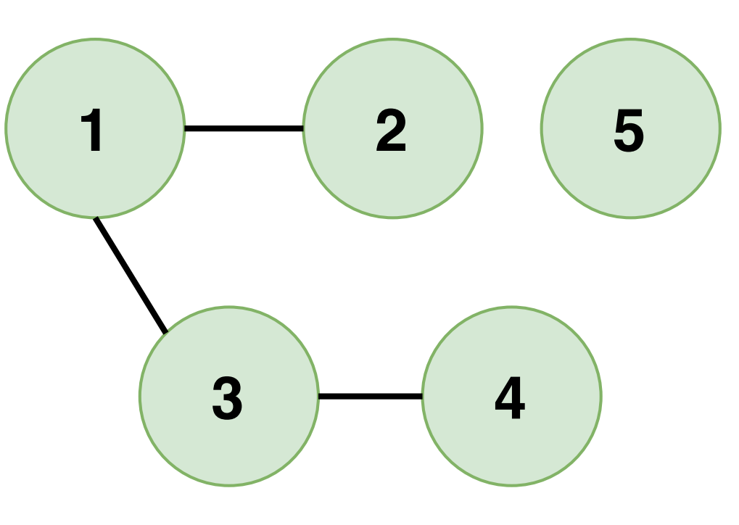
Union 3 and 4



Element	1	2	3	4	5
Parent	1	1	1	1	5

Parent of 3 is 1. So, parent of 4 will be 1.

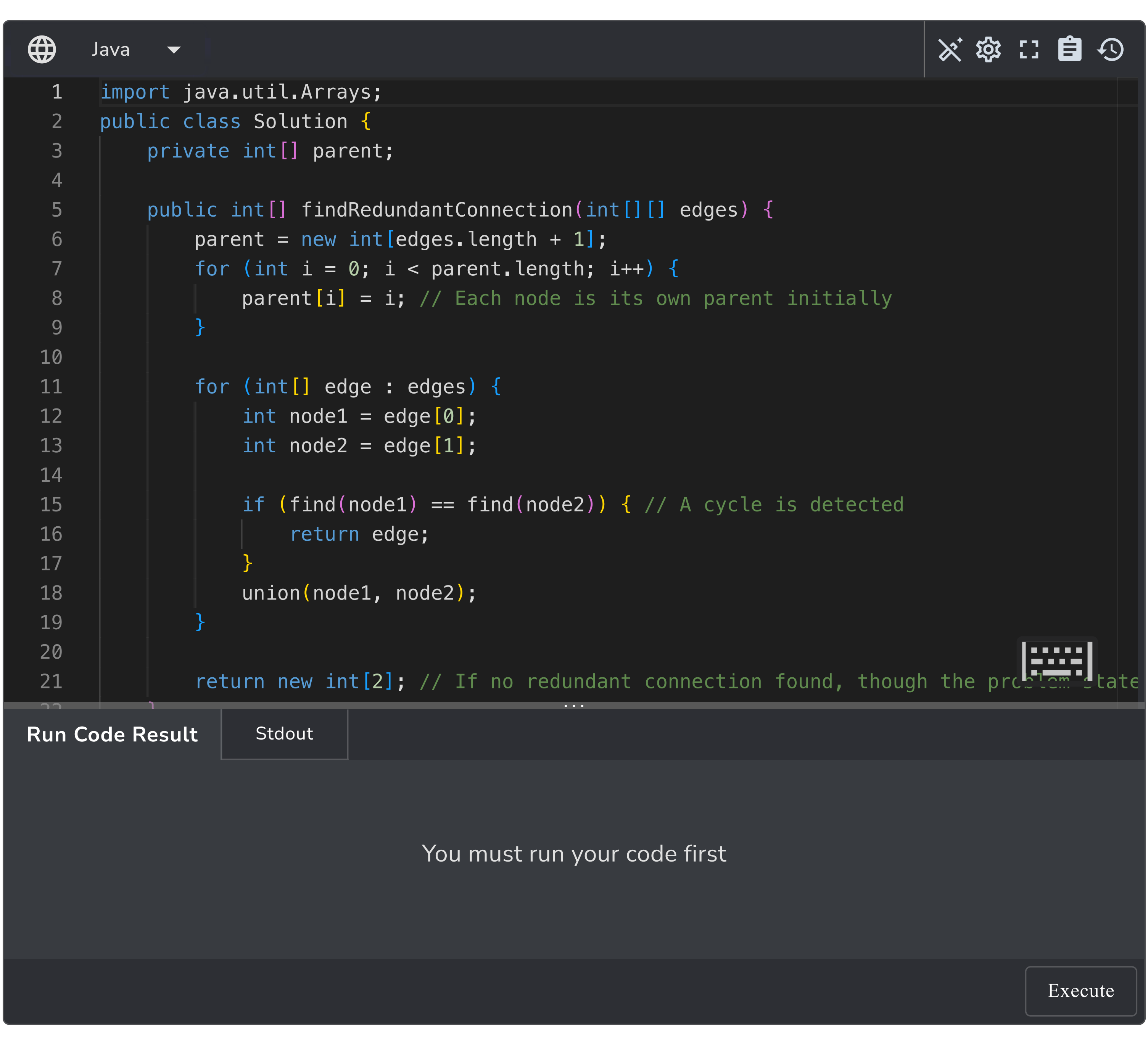
- Consider the edge `[1,4]`:
  - Find operations for both nodes return the same parent, indicating a cycle. We identify `[1,4]` as the redundant edge.



Element	1	2	3	4	5
Parent	1	1	1	1	5

1 and 4 has same parent. So, it will return [1, 4].

Code



```java
import java.util.Arrays;
public class Solution {
    private int[] parent;

    public int[] findRedundantConnection(int[][] edges) {
        parent = new int[edges.length + 1];
        for (int i = 0; i < parent.length; i++) {
            parent[i] = i; // Each node is its own parent initially
        }

        for (int[] edge : edges) {
            int node1 = edge[0];
            int node2 = edge[1];

            if (find(node1) == find(node2)) { // A cycle is detected
                return edge;
            }
            union(node1, node2);
        }

        return new int[2]; // If no redundant connection found, though the problem state
```

You must run your code first

Complexity Analysis:

- **Time Complexity**: The time complexity is O(N log N), where N is the number of nodes, due to the Union-Find operations with path compression over all edges.).
- **Space Complexity**: The space complexity is (O(N)), since we maintain a parent array of size (N) where (N) is the number of vertices.